

VL Reward Model 实验报告

csh / Codex assisted

2026-05-19

1 任务理解与最终交付

考核文档要求以 Qwen2-VL 或 InternVL2.5-VL 系列模型为 base model，在模型后端添加 score head，将其训练为多模态 reward model，并在 VLRewardBench 上完成评测。需要提交的核心内容包括：base model 在 VLRewardBench 上的 Score_{base} ，训练后的 reward model 权重，reward model 的 Score_{reward} ，以及包含数据处理、训练参数、实验发现和问题分析的 PDF 报告。

本实验最终完成了两个 backbone：

模型	Score_{base}	Score_{reward}	结论
InternVL2.5-2B	64.80%	71.21%	reward model 明显超过 base judge，也超过考核文档中 64.8% 的预先结果。
Qwen2-VL-7B-Instruct	43.14%	66.56%	reward model 显著超过原模型直接 judge。

这里的 Score_{base} 指原始 base model 在不训练 reward head 的情况下，直接读取图像、问题和两个候选回答并生成 A/B 判断； Score_{reward} 指训练后的 scalar reward model 分别给 A/B 回答打分，再比较分数得到的准确率。这两个设置的输入形式和归纳偏置不同，因此不能简单把 base judge 当作 reward head 的上界。base judge 同时看到两个回答，可以用完整生成式推理作比较；reward model 则需要把每个单独回答压缩成一个标量，更适合后续作为训练或推理阶段的 reward signal。

2 代码、环境与硬件约束

服务器工作目录：

/data4/ljl/csh/vl_reward_work

主要脚本：

- InternVL: `scripts/internvl_reward.py`
- Qwen2-VL: `scripts/qwen2vl_reward.py`

模型路径:

- InternVL2.5-2B: `/data2/ljl/csh/pretrained_model/InternVL2_5-2B`
- Qwen2-VL-7B-Instruct: `/data2/ljl/csh/pretrained_model/Qwen2-VL-7B-Instruct`

Python 环境:

- InternVL 环境: `/data3/ljl/miniconda3/envs/internvl/bin/python`
- Qwen 环境: `/data3/ljl/miniconda3/envs/qwen25vl/bin/python`

训练和评测均遵守“一次最多使用四张卡”的约束，主要使用 GPU 0-3，每张卡为 RTX 4090 24GB。训练时采用 DDP 四卡并行，评测时采用四个单卡 shard 并行跑完后合并 JSON 结果。

3 数据来源与处理流程

3.1 数据来源

训练数据没有使用 VLRewardBench。实际使用了两类偏好数据:

- RLHF-V: 服务器已有 `RLHF-V-Dataset.parquet`，经脚本处理后得到 2,533 条训练偏好对和 281 条验证偏好对。
- HuggingFaceH4 `rlaif-v_formatted`: 最初只下载了少数 `parquet shard`，后来为了提升覆盖面，补齐全量 13 个 `shard`，共 78,975 条原始偏好样本。

VLRewardBench 仅用于最终评测。评测集处理成 `data/processed/vlrewardbench_eval.jsonl`，共 1,247 条样本。

3.2 过滤与防泄漏

数据准备阶段从 VLRewardBench `eval` 文件中抽取 `query` 集合，作为训练数据过滤表。处理 H4 和 RLHF-V 时，如果训练候选样本的 `query` 与 VLRewardBench `query` 命中，则丢弃该训练样本。这个过滤只用于防止 `query` 级别泄漏，不使用 VLRewardBench 的 `preference label` 进行训练。

3.3 早期去重问题

最初的 H4 prepare 逻辑按 query 文本去重。这个选择后来被证明不合适：在多模态数据中，同一个文本问题可能对应不同图像，或者同一个 query 在不同图像上构成不同偏好判断。按 query 去重会把这些有效样本误删，导致训练集规模和视觉多样性都被压缩。

修正后加入 `--dedup-queries` 选项，并在最终数据构造中关闭 query 去重，改用 no-dedup 版本。最终用于训练的混合数据如下：

文件	样本数
data/processed_h4_30k_nodedup/train_pairs_h4.jsonl	29,000
data/processed_h4_30k_nodedup/val_pairs_h4.jsonl	1,000
data/processed_h4_30k_nodedup/train_pairs_mix.jsonl	31,533
data/processed_h4_30k_nodedup/val_pairs_mix.jsonl	1,281
data/processed/vlrewardbench_eval.jsonl	1,247

其中 `train_pairs_mix.jsonl` 为 H4 no-dedup 训练集与 RLHF-V 训练集的混合版本，`val_pairs_mix.jsonl` 为对应验证集。

4 方法设计

4.1 Base judge 评测

base judge 的目标是得到原模型的 Score_{base} 。对每条 VLRewardBench 样本，构造如下输入：图像、用户问题、Response A、Response B，以及要求模型“只输出 A 或 B”的裁判提示。模型生成文本后，用正则解析第一个独立的 A/B 作为预测；如果无法解析，记录 parse failure 并默认预测 A，以保证分母仍为完整 1,247 条样本。

InternVL 使用其原生 `model.chat` 接口，并设置 `max_num=4` 控制图像 tile 数量。Qwen2-VL 使用 `AutoProcessor.apply_chat_template` 构造单轮对话，再调用 `model.generate` 得到 A/B 输出。

Qwen base judge 过程中发现一个重要问题：候选回答较长时，`max_length=768` 可能截断掉提示末尾的“Answer with exactly one letter: A or B.”和 assistant generation prompt。此时 Qwen 不是在做比较，而是在续写候选回答，导致部分 raw output 形如数学推导、句子残片或其它非 A/B 文本。为避免把这种截断伪结果当作 base score，最终 Qwen base 使用：

- 每个候选回答最多保留 1,200 字符；
- `max_length=1536`；
- 四卡四 shard 完整评测；
- parse rate 达到 98.9%。

未保护的 max_length=768 版本得到 52.13%，但 parse rate 只有 81.4%，因此只作为诊断记录，不作为最终 Score_{base} 。

4.2 Reward model 结构

reward model 不再生成 A/B，而是对每个候选回答独立输出一个 scalar score。给定图像和问题 x ，候选回答 y ，模型输出最后一个有效 token 的隐藏状态 $h_{\text{last}}(x, y)$ ，再经过 LayerNorm + Linear head 得到：

$$s(x, y) = \text{Linear}(\text{LayerNorm}(h_{\text{last}}(x, y))).$$

一条偏好样本包含 chosen 和 rejected 两个回答。训练目标为 Bradley – Terry / pairwise logistic loss：

$$\mathcal{L} = -\log \sigma(s(x, y_{\text{chosen}}) - s(x, y_{\text{rejected}})).$$

推理时分别计算 s_A 和 s_B ，若 $s_A \geq s_B$ 则预测 A，否则预测 B。

4.3 LoRA 训练策略

两个 backbone 都冻结原始模型参数，只训练 LoRA adapter 和 scalar head。这样做有三个原因：

- 24GB 4090 上全参训练不可行，LoRA 可以显著降低显存与优化器状态开销；
- 保存的权重很小，便于上传 Hugging Face；
- reward head 训练主要需要调整语言侧偏好表示，低秩更新已经足够形成有效排序能力。

InternVL 使用已有模块结构，主要在语言模型的 wqkv, wo, w1, w2, w3 等线性层上加 LoRA。Qwen 环境没有可直接使用的 peft 和 bitsandbytes，因此实现了轻量 LoRALinear：冻结原 nn.Linear ，新增 $A \in \mathbb{R}^{r \times d_{\text{in}}}$ 与 $B \in \mathbb{R}^{d_{\text{out}} \times r}$ ，前向输出为原线性层输出加上 $\alpha/r \cdot B A x$ 。Qwen 的 LoRA 作用在 q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj。

5 训练配置

配置	InternVL2.5-2B Reward	Qwen2-VL-7B Reward
训练集	H4 30k no-dedup + RLHF-V mix	H4 30k no-dedup + RLHF-V mix
训练样本	31,533	31,533
验证样本	1,281	1,281
epoch	1	1

并行	4 GPU DDP	4 GPU DDP
max length	1536	768
图像设置	max_num=4	min_pixels=50176,max_pixels=200704
LoRA rank / alpha	16 / 32	8 / 16
LoRA 起始层	language layer 8	language layer 20
LoRA lr	4×10^{-5}	3×10^{-5}
Head lr	1.5×10^{-4}	1.5×10^{-4}
梯度累积	8	8
训练时间	3,201.7 s	2,360.8 s

InternVL2.5-2B 参数更小，因此可以使用更长文本长度和更高 LoRA rank。Qwen2-VL-7B 参数规模较大，为保证四张 4090 都能稳定跑满，不发生 OOM，采用 max_length=768、较低 rank，并只从第 20 层开始加 LoRA。

6 实验探索过程

6.1 Frozen head baseline

最早尝试是在 InternVL2.5-2B 上预计算特征，只训练一个 score head。这种做法显存和训练成本很低，但 VLRewardBench 只有 46.99%。结论是：base hidden representation 未针对偏好排序优化，仅靠线性头难以学习复杂的视觉回答偏好。

6.2 小规模 LoRA

随后在 InternVL 上用较小 H4/RLHF 混合集训练 LoRA reward，分数提升到 54.61%。这说明更新语言层比只训练头部有效，但数据规模、去重策略和上下文长度仍然限制明显。

6.3 Base judge pseudo-label 蒸馏

为解释“为什么 base judge 看起来比弱 reward head 高”，尝试把 base judge 在训练池上的 A/B 判断转成 pseudo preference，再训练 reward model。该方法得到 59.98%，比小规模 LoRA 高，但仍低于最终 H4 no-dedup 训练。这个实验说明 base judge 的判断可以提供一定信号，但单纯蒸馏 hard label 会继承 base judge 的偏差，而且 reward model 仍然缺少足够的真实偏好覆盖。

6.4 完整 H4 与 no-dedup 修正

最终提升来自三点叠加：补齐 H4 13 个 parquet shard；修复 query 去重导致的样本误删；提高 InternVL LoRA rank 和上下文长度。InternVL 最终达到 71.21%，明显超过原始 base judge。

6.5 Qwen2-VL 训练补齐

Qwen2-VL 不是一开始就完成的。由于服务器 Qwen 环境没有可直接用的 LoRA 依赖，补写了独立的 Qwen reward 脚本，包括 processor 构造、pairwise input、手写 LoRA、DDP 训练、reward eval、base judge eval 和结果合并。Qwen reward 最终达到 66.56%，高于严格 base judge 43.14%。

7 遇到的问题与解决

问题	表现	解决方式
InternVL forward 显存高	使用完整模型 forward 会产生 vocab logits，reward 训练显存压力大。	新增 hidden-state-only forward，绕过 lm head，只取最后 token hidden state。
H4 数据去重不当	按 query 去重导致同问题不同图像样本被误删。	增加 no-dedup 数据准备，最终使用 H4 30k no-dedup。
Qwen 环境缺依赖	无 peft/bitsandbytes，无法直接套用常规 LoRA。	实现手写 LoRALinear，只替换目标线性层。
Qwen base judge 截断	长回答截断末尾 A/B 指令，模型续写答案，parse rate 低。	response clipping + max_length=1536，最终 parse rate 98.9%。
4090 24GB 显存限制	Qwen2-VL-7B 无法使用过长上下文和过大 rank。	Qwen 使用 rank 8、后 8 层 LoRA、max_length=768。
评测耗时	VLRewardBench 生成式 judge 和 reward eval 都较慢。	四卡按 shard 并行，最终用 merge 脚本合并。

8 最终结果

8.1 VLRewardBench

模型	Base score	Reward score	提升
InternVL2.5-2B	64.80%	71.21%	+6.42
Qwen2-VL-7B-Instruct	43.14%	66.56%	+23.42

考核文档中给出的预先结果为 Qwen2-VL-7B-Reward 51.2%、InternVL2.5-2B-Reward 64.8%。本实验最终两个 reward model 均超过该参考线。

8.2 训练过程指标

模型	train acc	val acc	val margin	step
InternVL2.5-2B Reward	68.89%	74.24%	1.1585	986
Qwen2-VL-7B Reward	61.34%	65.42%	0.3812	986

InternVL 的验证 margin 更高，说明 H4/RLHF 混合数据在较小模型上形成了更稳定的偏好排序边界。Qwen 的训练精度和验证精度较低，但最终 VLRewardBench reward 分数仍较高，说明即使训练设置更保守，LoRA + scalar head 已经学到比 direct judge 更有用的 reward 表示。

9 文件与权重

项目	路径或 repo
GitHub 代码	https://github.com/Serenananah/VL-Reward-Model.git
InternVL HF repo	Hanzi7na/internvl25-2b-vlreward-lora
Qwen HF repo	Hanzi7na/qwen2vl-7b-vlreward-lora
InternVL best checkpoint	outputs/internvl25_2b_lora_reward_h4mix30k_r16_l8_len1536_no_logits_4g/reward_lora_best.pt
Qwen best checkpoint	outputs/qwen2vl7b_lora_reward_h4mix30k_r8_l20_len768_4g/reward_lora_best.pt
InternVL base eval	results/base_judge_internvl25_2b_max4_rerun_20260519.json
Qwen base eval	results/base_judge_qwen2vl7b_len1536_clip1200_rerun_20260519.json
InternVL reward eval	results/internvl25_2b_lora_reward_h4mix30k_r16_l8_len1536_no_logits_4g_vlrewardbench.json
Qwen reward eval	results/qwen2vl7b_lora_reward_h4mix30k_r8_l20_len768_4g_vlrewardbench.json

Hugging Face 权重文件是 PyTorch checkpoint bundle，包含 LoRA 可训练参数、score head state、LoRA config 和训练参数。为了避免 GitHub 仓库过大，代码仓库不上传 checkpoint，权重单独放在两个 HF repo。

10 复现流程

本节给出从环境、数据准备、训练到评测的复现路径。实际服务器路径为 `/data4/ljl/csh/vl_reward_work`；如果在其它机器复现，只需要保持 JSONL 字段格式一致，并把模型路径替换成自己的本地路径。

10.1 环境安装

两个模型使用了不同的已验证环境。InternVL 最终训练环境为 /data3/ljl/miniconda3/envs/internvl，主要版本为 torch==2.1.2+cu118, torchvision==0.16.2+cu118, transformers==4.37.2, accelerate==0.34.2, pandas==2.3.3, pyarrow==21.0.0, Pillow==11.3.0, tqdm==4.67.3, timm==0.9.12, einops==0.6.1。

Qwen2-VL 最终训练环境为 /data3/ljl/miniconda3/envs/qwen25vl，主要版本为 torch==2.5.1+cu121, torchvision==0.20.1+cu121, transformers==5.1.0, accelerate==1.12.0, qwen-vl-utils==0.0.8, Pillow==12.1.0, tqdm==4.67.3, numpy==1.26.4。

代码仓库中补充了 requirements.txt。由于两个模型的测试环境使用了不同的 torch/transformers 版本，最稳妥的做法是建立两个 conda 环境，而不是强行把所有版本装进同一个环境。示例：

```
conda create -n internvl_reward python=3.9 -y
conda activate internvl_reward
pip install torch==2.1.2 torchvision==0.16.2 --index-url https://download.pytorch.org/whl/cu118
pip install transformers==4.37.2 accelerate==0.34.2 pandas==2.3.3 pyarrow==21.0.0 \
    Pillow==11.3.0 tqdm==4.67.3 timm==0.9.12 einops==0.6.1 sentencepiece==0.1.99

conda create -n qwen2vl_reward python=3.10 -y
conda activate qwen2vl_reward
pip install torch==2.5.1 torchvision==0.20.1 --index-url https://download.pytorch.org/whl/cu121
pip install transformers==5.1.0 accelerate==1.12.0 qwen-vl-utils==0.0.8 \
    Pillow==12.1.0 tqdm==4.67.3 numpy==1.26.4 safetensors==0.7.0
```

10.2 数据准备

原始数据需要先处理成偏好对 JSONL，每行至少包含 image_path, query, response_a, response_b, label。其中 label=0 表示 A 更优，label=1 表示 B 更优。原实验处理流程为：

```
python scripts/internvl_reward.py prepare \
    --rlhf-v-parquet data/raw/RLHF-V-Dataset.parquet \
    --vlrewardbench-jsonl data/processed/vlrewardbench_eval.jsonl \
    --out-dir data/processed

python scripts/internvl_reward.py prepare-h4 \
    --h4-dir data/raw/rlaif_h4 \
    --vlrewardbench-jsonl data/processed/vlrewardbench_eval.jsonl \
    --out-dir data/processed_h4_30k_nodedup \
    --max-train 29000 --max-val 1000

python scripts/internvl_reward.py mix-jsonl \
    --inputs data/processed_h4_30k_nodedup/train_pairs_h4.jsonl data/processed/train_pairs.jsonl \
    --out data/processed_h4_30k_nodedup/train_pairs_mix.jsonl
```


10.3 训练命令

InternVL2.5-2B reward 训练命令如下：

```
CUDA_VISIBLE_DEVICES=0,1,2,3 torchrun --nproc_per_node 4 scripts/internvl_reward.py train-lora \
--model-path /data2/ljl/csh/pretrained_model/InternVL2_5-2B \
--train-jsonl data/processed_h4_30k_nodedup/train_pairs_mix.jsonl \
--val-jsonl data/processed_h4_30k_nodedup/val_pairs_mix.jsonl \
--out-dir outputs/internvl25_2b_lora_reward_h4mix30k_r16_l8_len1536_no_logits_4g \
--max-num 4 --max-length 1536 --lora-rank 16 --lora-alpha 32 --lora-layer-start 8 \
--grad-accum 8 --epochs 1
```

Qwen2-VL-7B reward 训练命令如下：

```
CUDA_VISIBLE_DEVICES=0,1,2,3 torchrun --nproc_per_node 4 scripts/qwen2vl_reward.py train \
--model-path /data2/ljl/csh/pretrained_model/Qwen2-VL-7B-Instruct \
--train-jsonl data/processed_h4_30k_nodedup/train_pairs_mix.jsonl \
--val-jsonl data/processed_h4_30k_nodedup/val_pairs_mix.jsonl \
--out-dir outputs/qwen2vl7b_lora_reward_h4mix30k_r8_l20_len768_4g \
--max-length 768 --min-pixels 50176 --max-pixels 200704 \
--lora-rank 8 --lora-alpha 16 --lora-layer-start 20 --grad-accum 8 --epochs 1
```

10.4 评测命令

VLRewardBench 评测采用四卡分片方式。以 InternVL reward 为例，每张卡运行一个 shard：

```
for i in 0 1 2 3; do
    CUDA_VISIBLE_DEVICES=$i python scripts/internvl_reward.py eval-lora-reward \
        --model-path /data2/ljl/csh/pretrained_model/InternVL2_5-2B \
        --jsonl data/processed/vlrewardbench_eval.jsonl \
        --checkpoint outputs/internvl25_2b_lora_reward_h4mix30k_r16_l8_len1536_no_logits_4g/
reward_lora_best.pt \
        --out results/internvl25_reward_shard${i}.json \
        --device cuda:0 --max-num 4 --max-length 1536 \
        --num-shards 4 --shard-index ${i} &
done
wait

python scripts/internvl_reward.py merge-eval-json \
    --inputs results/internvl25_reward_shard0.json results/internvl25_reward_shard1.json \
    results/internvl25_reward_shard2.json results/internvl25_reward_shard3.json \
    --out results/internvl25_reward_merged.json
```

Qwen reward 和 base judge 的评测入口分别是 `scripts/qwen2vl_reward.pyeval` 与 `scripts/qwen2vl_reward.pybase-judge-eval`；InternVL base judge 入口是 `scripts/internvl_reward.pybase-judge-eval`。代码仓库中的 `results/*.json` 已经是最终 merged 版本。

11 结论与后续改进

本实验完成了两个多模态 reward model 的训练、评测和权重保存。关键结论是：reward model 的效果高度依赖偏好数据规模、数据清洗、LoRA 覆盖范围和显存友好的 hidden-state-only 实现。最终 InternVL2.5-2B reward 达到 71.21%，Qwen2-VL-7B reward 达到 66.56%。

后续如果继续提升，可以优先尝试四个方向：第一，对 Qwen 使用更长训练上下文和更高 LoRA rank；第二，针对 wildvision-battle 等开放偏好来源做 hard negative mining；第三，对 H4、RLHF-V 和不同 VLRewardBench source 的分布差异做 reweighting；第四，引入 base judge 的 reasoning 或 soft preference margin，而不是只蒸馏 hard A/B label。